TEMA 3 · PROPIEDADES DE LOS LENGUAJES REGULARES

Algoritmos para autómatas

Vacío o no vacío, finito o infinito e igualdad de lenguajes

Ejemplos y recorridos paso a paso.

Nuria Rodríguez Barroso

Los problemas que vamos a decidir

01 **Vacío o no vacío.** ¿Existe alguna palabra aceptada?

02 **Finitud.** ¿Hay un número finito o infinito de palabras aceptadas?

03 **Igualdad.** ¿Aceptan dos autómatas exactamente las mismas palabras?

> Trabajaremos con AFD completos sobre $\Sigma = \{0, 1\}$, sin transiciones ε.

Vacío o no vacío: el criterio

$$L(M) \neq \varnothing \iff \exists q \in F : \ q \text{ es accesible desde } q_0$$

Calculamos los estados alcanzables desde el inicial mediante un recorrido por el grafo.

Si alguno es final, tenemos una palabra aceptada. Si ninguno es final, el lenguaje es vacío.

> Si el propio inicial es final, la palabra ε ya demuestra que el lenguaje no es vacío.

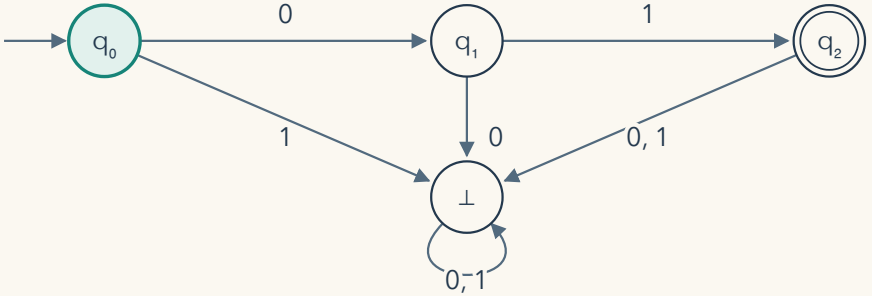
Vacío o no vacío: procedimiento

```
R ← {q₀}
pendientes ← [q₀]
mientras haya estados pendientes:
    extraer el primero de pendientes: q
    para a = 0 y después a = 1:
        añadir a R y al final de pendientes los sucesores nuevos
si R ∩ F = ∅: devolver «vacío»
si no: devolver «no vacío»
```

> Un sucesor es nuevo si aún no pertenece a R. La cola pendientes se atiende de izquierda a derecha (BFS).

Coste: $O(|Q| + |E|)$, con E el conjunto de transiciones.

Ejemplo 1: inicialización



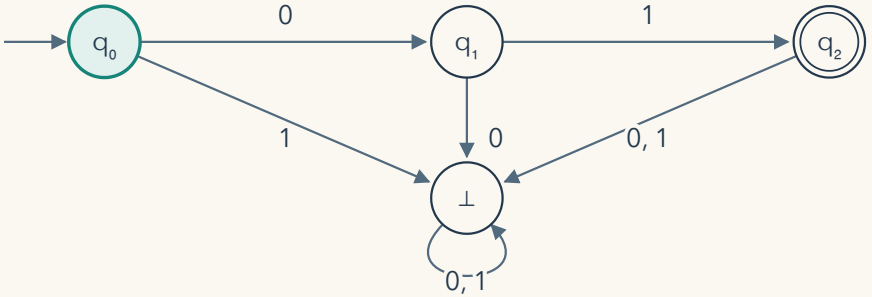
R = $\{q_0\}$

pendientes = $[q_0]$

F = $\{q_2\}$

Inicializamos R y pendientes con el estado inicial. La cola se atiende de izquierda a derecha.

Ejemplo 1: extraemos q0



q = q_0

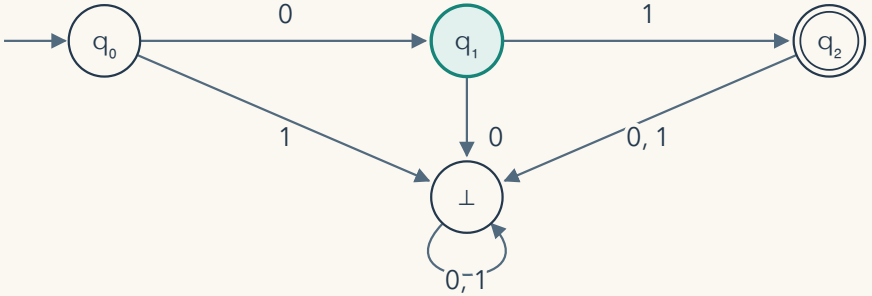
R = $\{q_0, q_1, \perp\}$

pendientes = $[q_1, \perp]$

F = $\{q_2\}$

Extraemos q_0: $[q_0] \rightarrow []$. Con 0 llegamos a q_1 y con 1 a ⊥. Añadimos los nuevos: $\{q_1, \perp\}$.

Ejemplo 1: extraemos q1



q = q_1

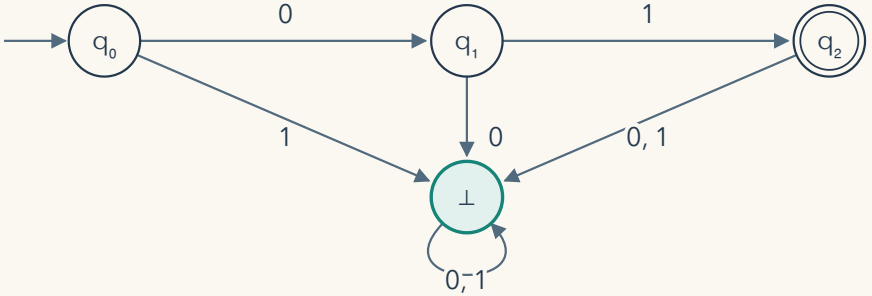
R = $\{q_0, q_1, \perp, q_2\}$

pendientes = $[\perp, q_2]$

F = $\{q_2\}$

Extraemos q_1: $[q_1, \perp] \rightarrow [\perp]$. Con 0 llegamos a $\perp$ y con 1 a q_2. Añadimos los nuevos: $\{q_2\}$.

Ejemplo 1: extraemos ⊥



q = ⊥

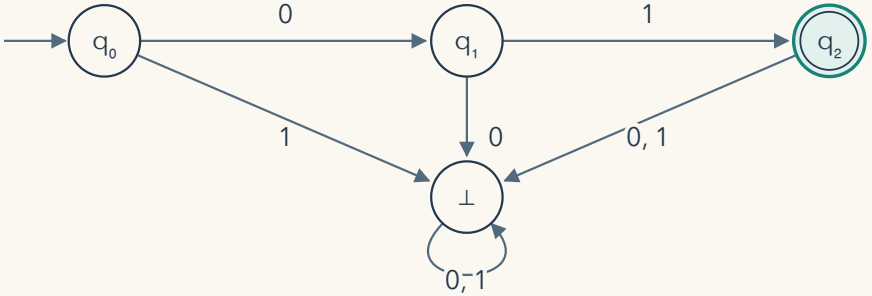
R = $\{q_0, q_1, \perp, q_2\}$

pendientes = $[q_2]$

F = $\{q_2\}$

Extraemos ⊥: $[\perp, q_2] \rightarrow [q_2]$. Con 0 llegamos a ⊥ y con 1 a ⊥. Ya están en R. No añadimos ni repetimos estados.

Ejemplo 1: extraemos q2



q = q_2

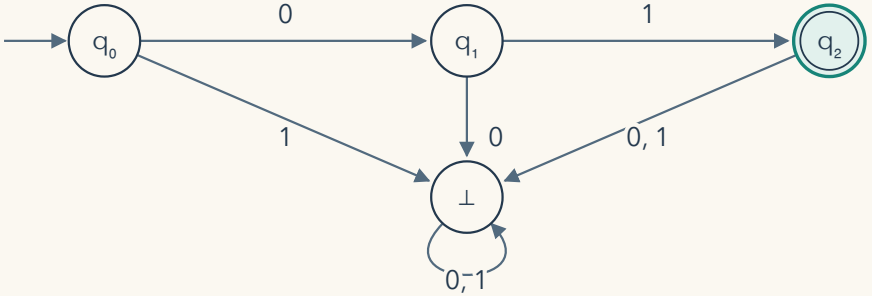
R = $\{q_0, q_1, \perp, q_2\}$

pendientes = []

F = $\{q_2\}$

Extraemos q_2: $[q_2] \rightarrow []$. Con 0 llegamos a ⊥ y con 1 a ⊥. Ya están en R. No añadimos ni repetimos estados.

Ejemplo 1: comprobación final



R = $\{q_0, q_1, \perp, q_2\}$

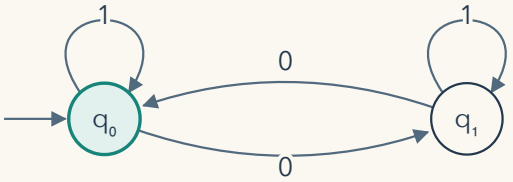
pendientes = []

F = $\{q_2\}$

R ∩ F = $\{q_2\} \neq \varnothing$

pendientes = []: termina el bucle. Ahora comprobamos la intersección. No vacío. La palabra 01 lleva al final q_2.

Ejemplo 2: inicialización





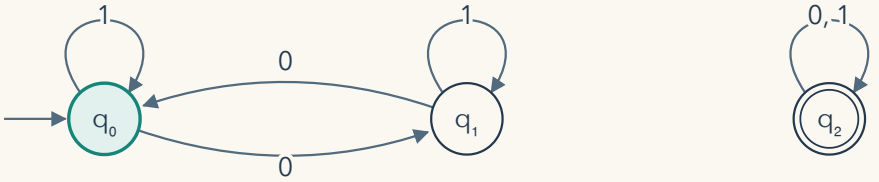
R = $\{q_0\}$

pendientes = $[q_0]$

F = $\{q_2\}$

Inicializamos R y pendientes con el estado inicial. La cola se atiende de izquierda a derecha.

Ejemplo 2: extraemos q0



q = q_0

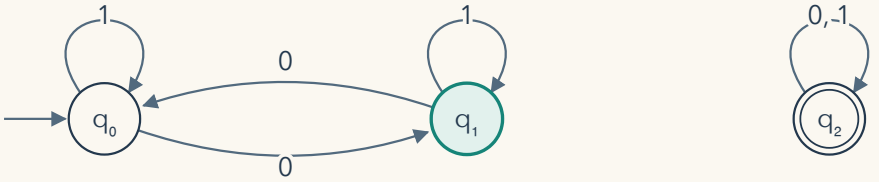
R = $\{q_0, q_1\}$

pendientes = $[q_1]$

F = $\{q_2\}$

Extraemos q_0: $[q_0] \rightarrow []$. Con 0 llegamos a q_1 y con 1 a q_0. Añadimos los nuevos: $\{q_1\}$.

Ejemplo 2: extraemos q1



q = q_1

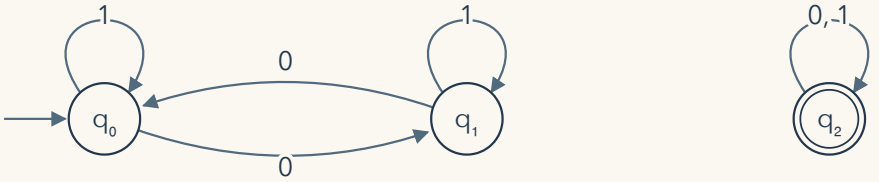
R = $\{q_0, q_1\}$

pendientes = []

F = $\{q_2\}$

Extraemos q_1: $[q_1] \rightarrow []$. Con 0 llegamos a q_0 y con 1 a q_1. Ya están en R. No añadimos ni repetimos estados.

Ejemplo 2: comprobación final



R = $\{q_0, q_1\}$

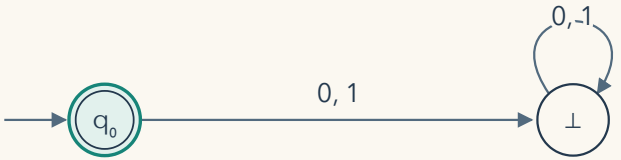
pendientes = []

F = $\{q_2\}$

R ∩ F = ∅

pendientes = []: termina el bucle. Ahora comprobamos la intersección. Vacío.

Ejemplo 3: inicialización



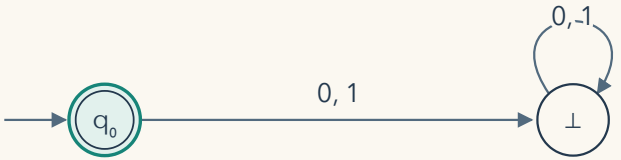
R = $\{q_0\}$

pendientes = $[q_0]$

F = $\{q_0\}$

Inicializamos R y pendientes con el estado inicial. La cola se atiende de izquierda a derecha.

Ejemplo 3: extraemos q0



q = q_0

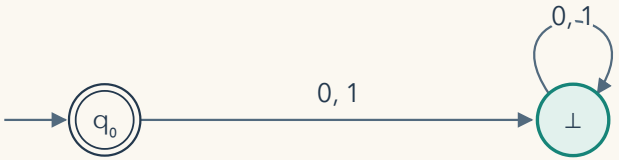
R = $\{q_0, \bot\}$

pendientes = $[\bot]$

F = $\{q_0\}$

Extraemos q_0: $[q_0] \rightarrow []$. Con 0 llegamos a ⊥ y con 1 a ⊥. Añadimos los nuevos: $\{\bot\}$.

Ejemplo 3: extraemos ⊥



q = ⊥

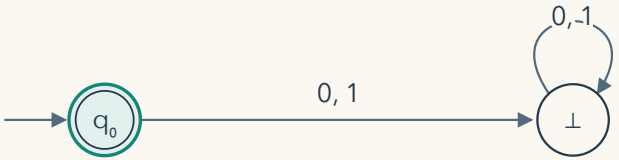
R = $\{q_0, \perp\}$

pendientes = []

F = $\{q_0\}$

Extraemos ⊥: [⊥] → []. Con 0 llegamos a ⊥ y con 1 a ⊥. Ya están en R. No añadimos ni repetimos estados.

Ejemplo 3: comprobación final



R = $\{q_0, \bot\}$

pendientes = []

F = $\{q_0\}$

R ∩ F = $\{q_0\} \neq \varnothing$

pendientes = []: termina el bucle. Ahora comprobamos la intersección. No vacío. L = {ε}: la palabra vacía está aceptada.

Ciclos en recorridos de aceptación

Un ciclo permite repetir una parte del recorrido tantas veces como queramos.

$$q_0 \xrightarrow{u} p \xrightarrow{v} p \xrightarrow{w} f \in F, \qquad |v| > 0$$

$$uv^k w \in L(M) \quad \text{para todo } k \geq 0$$

> El ciclo debe ser accesible desde el inicial y permitir llegar a un final. Cada repetición aumenta la longitud de la palabra.

Los estados útiles

R: estados alcanzables desde el inicial.

C: estados desde los que se puede llegar a algún final.

$$U = R \cap C$$

L(M) es infinito si y solo si el subgrafo de estados útiles contiene un ciclo.

> Obtenemos C recorriendo el grafo en sentido inverso, desde todos los estados finales.

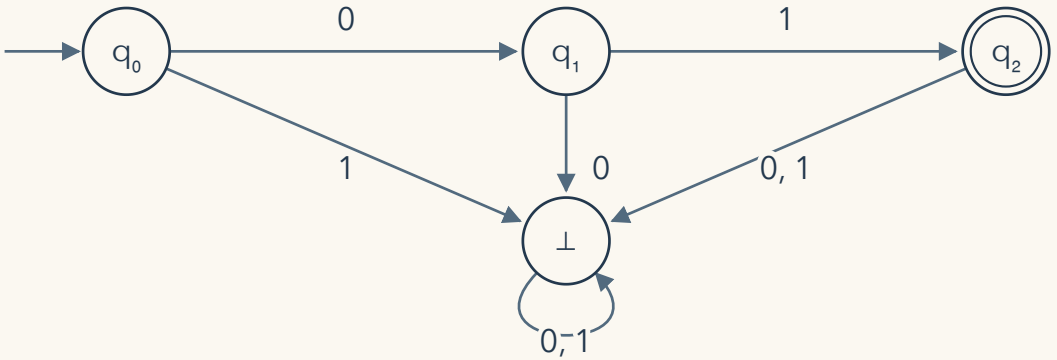
Finito o infinito: procedimiento

```
R ← alcanzables desde q₀
C ← alcanzables desde F en el grafo inverso
U ← R ∩ C
Gᵤ ← subgrafo que conserva solo estados de U
si Gᵤ tiene un ciclo: devolver «infinito»
si no: devolver «finito»
```

> Podemos detectar ciclos con DFS y estados en exploración. Si U = ∅, el lenguaje es vacío y, por tanto, finito.

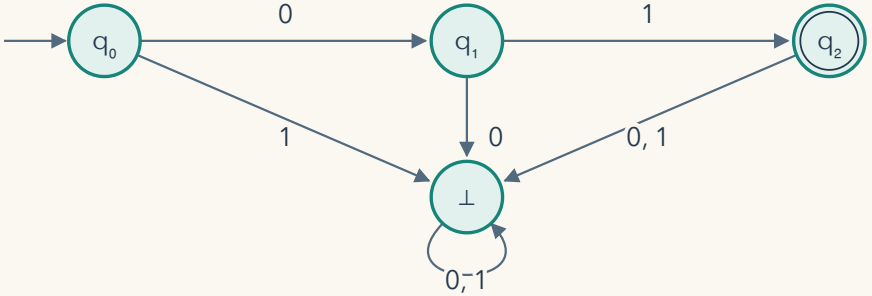
Coste: O(|Q| + |E|). Sin ciclos útiles, un camino aceptado no repite estados.

Un autómata con ciclos



¿Es infinito su lenguaje? El sumidero tiene un bucle, pero ningún recorrido que pase por él termina aceptado.

Paso 1: calculamos R

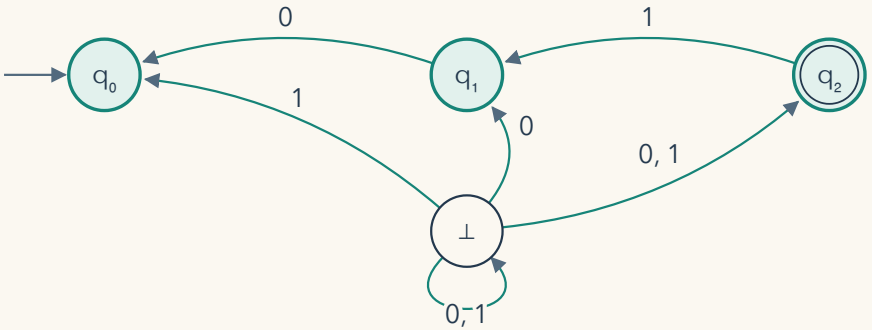


Inicial = q_0

R = $\{q_0, q_1, \perp, q_2\}$

Calculamos de una vez todos los estados accesibles desde q_0 siguiendo las transiciones del autómata.

Paso 2: calculamos C

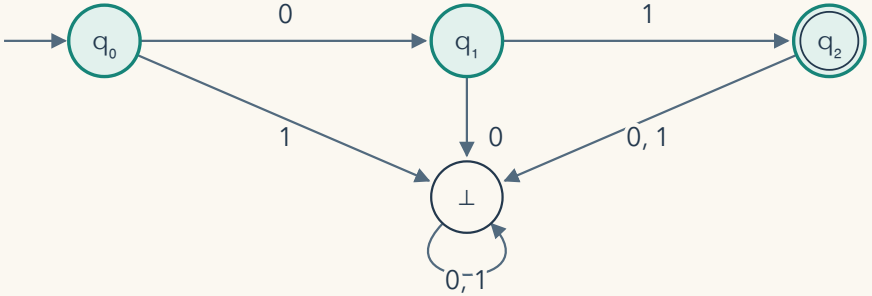


Finales = $\{q_2\}$

C = $\{q_2, q_1, q_0\}$

Invertimos todas las transiciones y calculamos de una vez los estados accesibles desde los finales.

Paso 3: calculamos U



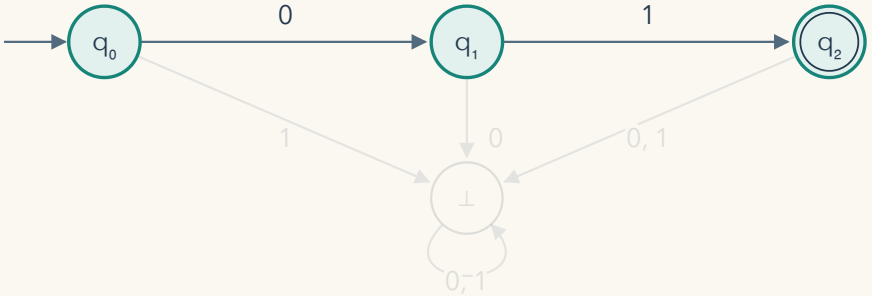
$\mathbf{R} = \{q_0, q_1, \perp, q_2\}$

$\mathbf{C} = \{q_2, q_1, q_0\}$

$\mathbf{U} = \mathbf{R} \cap \mathbf{C} = \{q_0, q_1, q_2\}$

Calculamos la intersección. ⊥ no pertenece a C y queda fuera de U.

Paso 4: construimos G_u



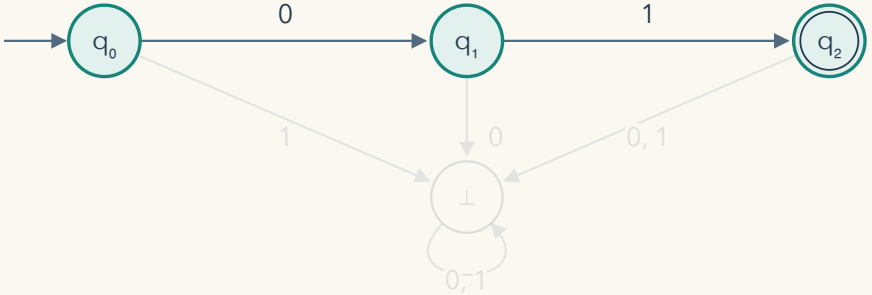
U = $\{q_0, q_1, q_2\}$

G_u: solo estados de U

Se conservan los arcos entre ellos.

Atenuamos ⊥ y los arcos que lo tocan. El dibujo destacado es el subgrafo G_u.

Paso 5: comprobamos los ciclos



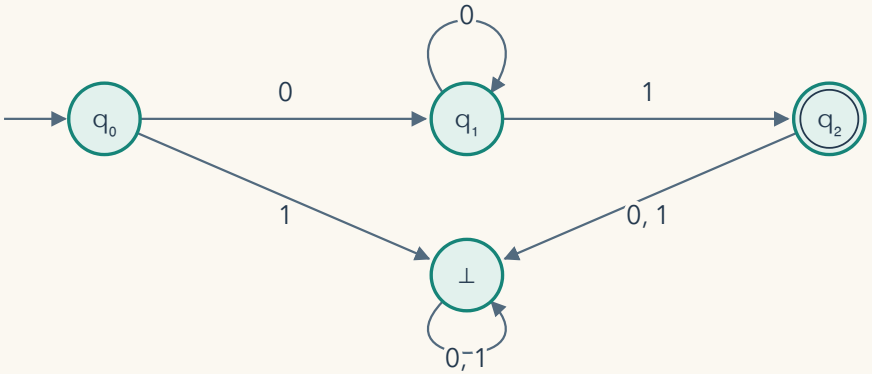
G_u: $q_0 \to q_1 \to q_2$

¿Tiene un ciclo? No

Devolver: «finito»

Ningún camino de G_u vuelve a un estado anterior. Ejecutamos la rama «si no». L = {01}.

Lenguaje infinito con ciclo útil

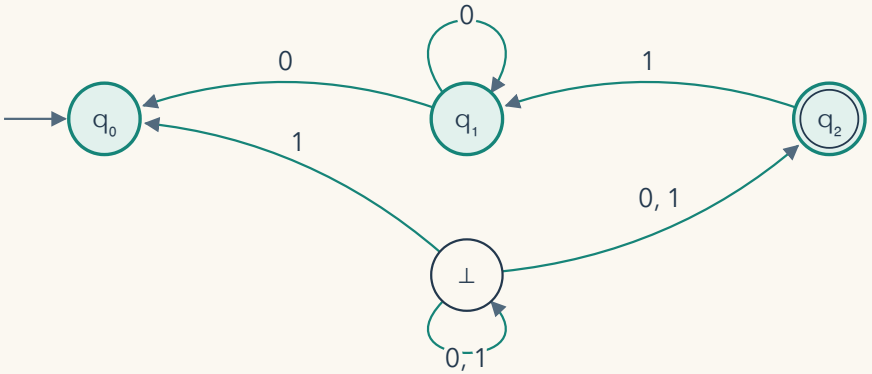


Inicial = q_0

R = $\{q_0, q_1, \perp, q_2\}$

Calculamos todos los estados accesibles desde el inicial en un solo paso.

Ejemplo infinito: calculamos C

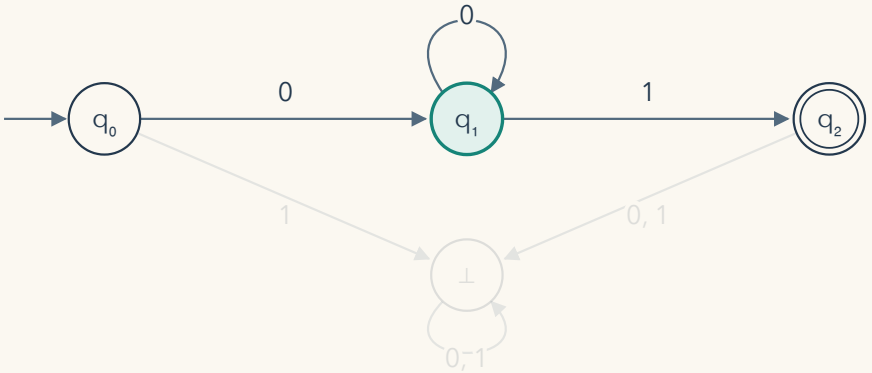


Finales = $\{q_2\}$

C = $\{q_2, q_1, q_0\}$

Con todas las transiciones invertidas, calculamos los accesibles desde el conjunto de estados finales en un solo paso.

Ejemplo infinito: calculamos U y G_u

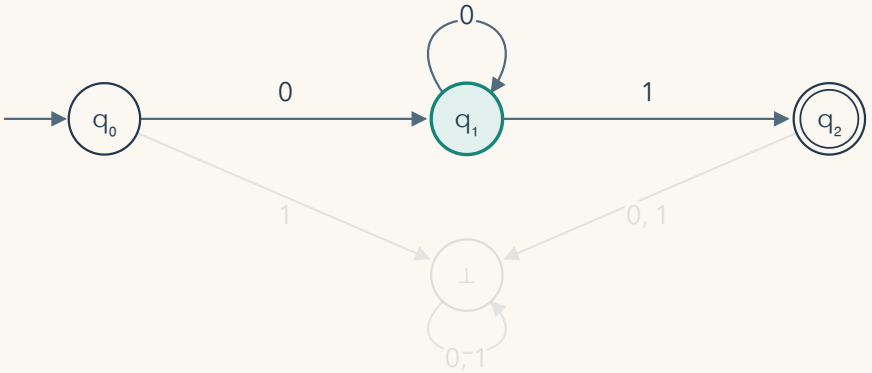


U = R ∩ C = $\{q_0, q_1, q_2\}$

G_u: estados de U y arcos entre ellos.

Conservamos el subgrafo indicado por U. Los estados atenuados quedan fuera.

Ejemplo infinito: comprobamos los ciclos

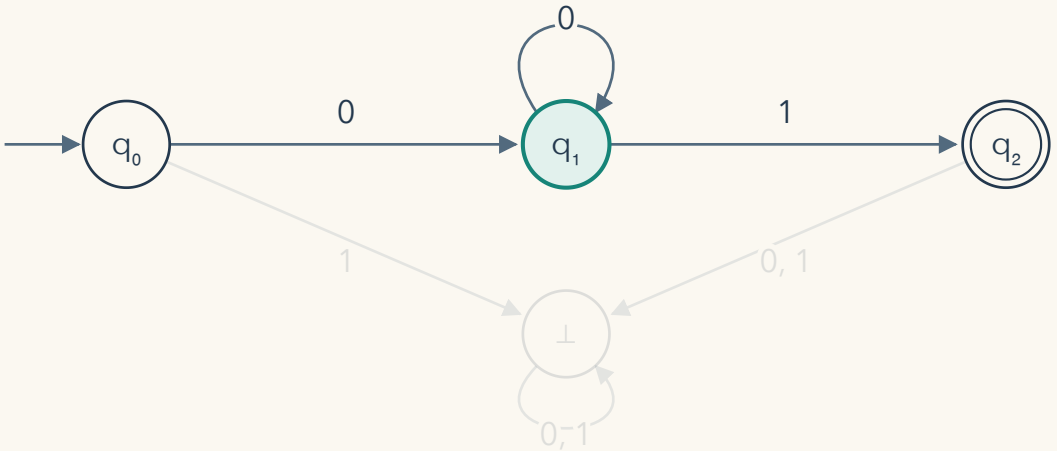


¿G_u tiene un ciclo? Sí: $q_1 \to q_1$ con 0

Devolver: «infinito»

El bucle de q_1 pertenece a G_u.

Una familia infinita de palabras

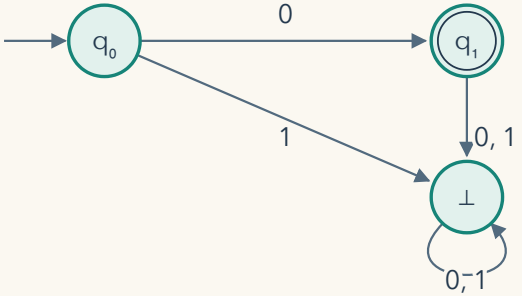


$$0 \cdot 0^k \cdot 1, \quad k \geq 0$$

01, 001, 0001, 00001, …

Cada vuelta añade un 0. Todas las palabras terminan en q_2 y sus longitudes son distintas.

Ciclo inaccesible: calculamos R



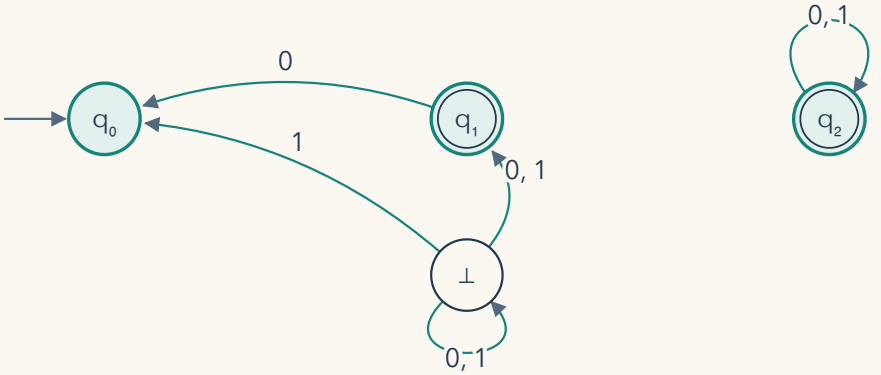


Inicial = q_0

R = $\{q_0, q_1, \perp\}$

Calculamos todos los estados accesibles desde el inicial en un solo paso.

Ciclo inaccesible: calculamos C

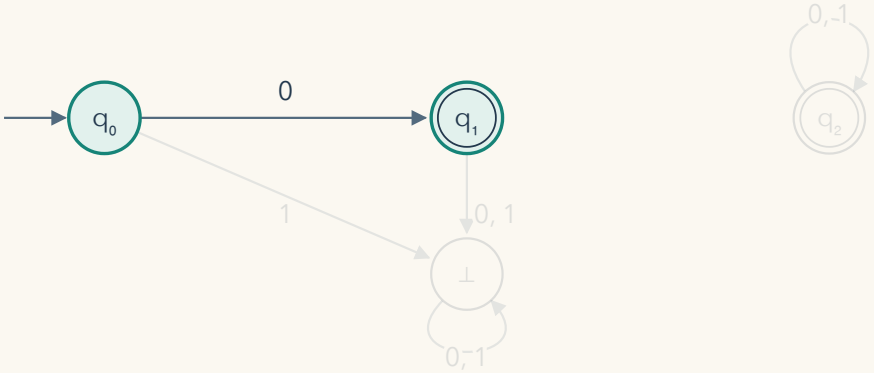


Finales = $\{q_1, q_2\}$

C = $\{q_1, q_2, q_0\}$

Con todas las transiciones invertidas, calculamos los accesibles desde el conjunto de estados finales en un solo paso.

Ciclo inaccesible: calculamos U y G_u

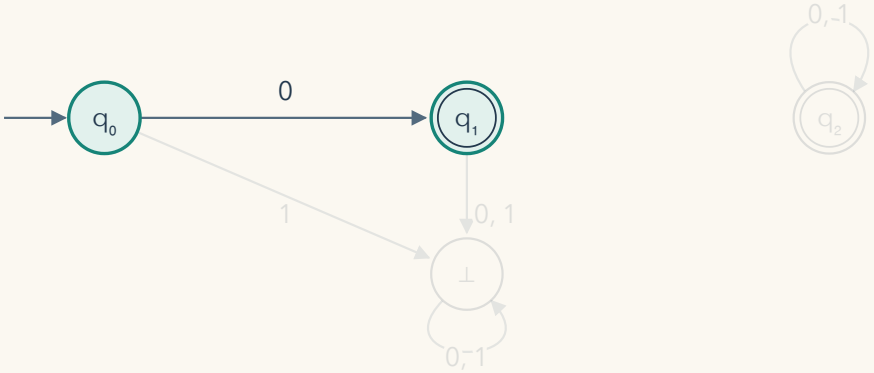


U = R ∩ C = $\{q_0, q_1\}$

G_u: estados de U y arcos entre ellos.

Conservamos el subgrafo indicado por U. Los estados atenuados quedan fuera.

Ciclo inaccesible: comprobamos los ciclos



¿G_u tiene un ciclo? No

Devolver: «finito»

El ciclo de q_2 queda fuera de G_u porque q_2 no pertenece a R. L = {0}.

El lenguaje que detecta las diferencias

$$L(P) = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$$

La primera parte contiene las palabras que M_1 acepta y M_2 rechaza. La segunda contiene las que M_2 acepta y M_1 rechaza.

$$L(M_1) = L(M_2) \iff L(P) = \varnothing$$

> Los complementarios se toman respecto a Σ*, con el mismo alfabeto Σ en ambos autómatas. Comprobamos si el lenguaje del producto es vacío o no vacío.

Estados finales del producto

$$\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$$

Cada transición lee el mismo símbolo en los dos autómatas.

Una pareja es final si exactamente uno de sus componentes es final.

Si ambos son finales, los dos aceptan. Si ninguno es final, los dos rechazan. En ninguno de esos casos hay una diferencia.

Igualdad de lenguajes: procedimiento

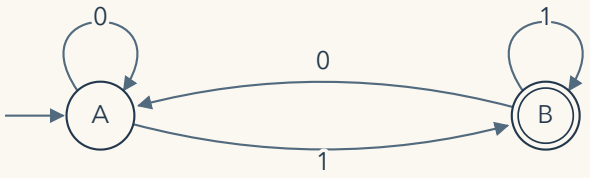
```
P ← producto de M₁ y M₂
inicial de P ← (q₀₁, q₀₂)
finales de P ← parejas con exactamente un final
si L(P) es vacío: devolver «iguales»
si no: devolver «distintos» y una palabra testigo
```

> Usamos AFD completos sobre el mismo alfabeto. Si faltan transiciones, añadimos un sumidero. Para AFN, determinizamos primero.

A lo sumo $|Q_1|\cdot|Q_2|$ parejas. Si la pareja inicial es final del producto, ε ya distingue los lenguajes.

Dos autómatas para comparar

M1: termina en 1

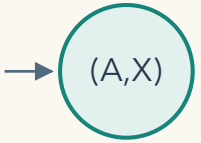


M2: contiene algún 1



> Ambos aceptan 1 y 01. Eso todavía no demuestra que sus lenguajes sean iguales.

Producto: la pareja inicial



Comenzamos en (A, X). Ninguno de sus componentes es final.

> Construimos las parejas alcanzables siguiendo los arcos de los dos dibujos.

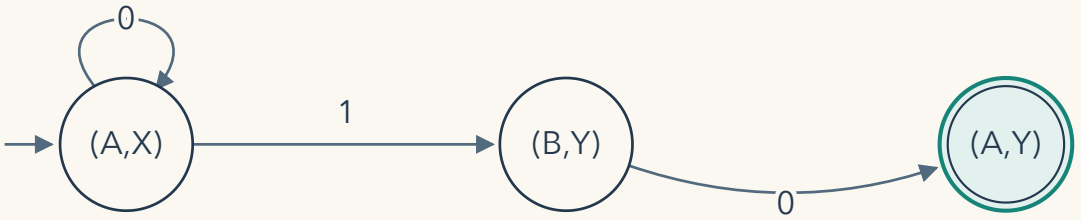
Producto: transiciones desde (A, X)



Con 0, ambos permanecen en su estado: (A, X). Con 1, llegan a (B, Y).

> (B, Y) no es final del producto: B e Y son finales a la vez.

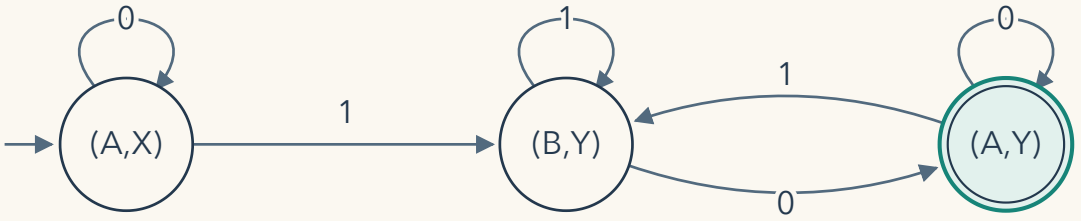
Producto: aparece (A, Y)



Desde (B, Y), el símbolo 0 lleva a A en M_1 y a Y en M_2.

> (A, Y) es final del producto: A no es final, pero Y sí.

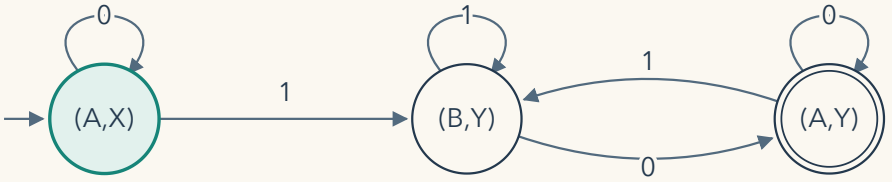
Producto: dibujo completo



Completamos los arcos de (B, Y) y (A, Y) con los símbolos 0 y 1.

> La pareja (B, X) no es alcanzable. El dibujo contiene todas las parejas que descubrimos desde la inicial.

Recorrido del producto: inicialización



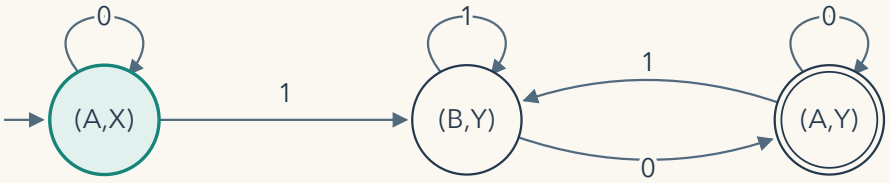
R = {(A, X)}

pendientes = [(A, X)]

F = {(A, Y)}

Inicializamos R y pendientes con el estado inicial. La cola se atiende de izquierda a derecha.

Recorrido del producto: extraemos (A, X)



q = (A, X)

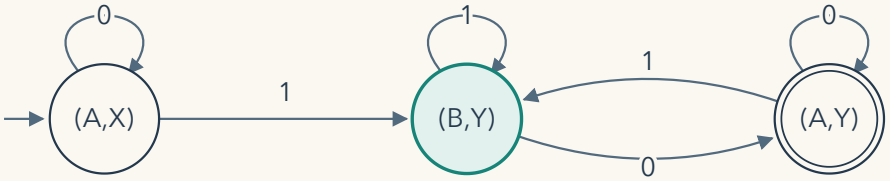
R = {(A, X), (B, Y)}

pendientes = [(B, Y)]

F = {(A, Y)}

Extraemos (A, X): [(A, X)] → []. Con 0 llegamos a (A, X) y con 1 a (B, Y). Añadimos los nuevos: {(B, Y)}.

Recorrido del producto: extraemos (B, Y)



q = (B, Y)

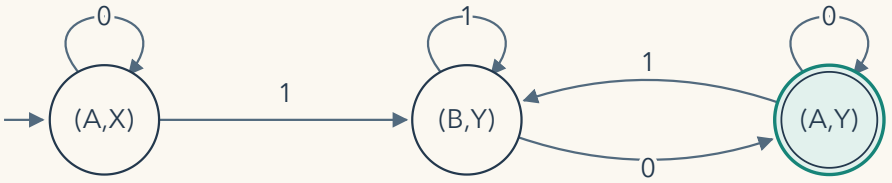
R = {(A, X), (B, Y), (A, Y)}

pendientes = [(A, Y)]

F = {(A, Y)}

Extraemos (B, Y): [(B, Y)] → []. Con 0 llegamos a (A, Y) y con 1 a (B, Y). Añadimos los nuevos: {(A, Y)}.

Recorrido del producto: extraemos (A, Y)



q = (A, Y)

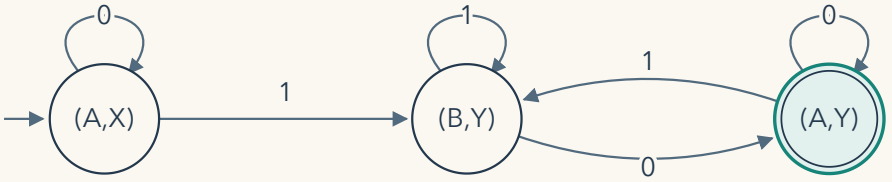
R = {(A, X), (B, Y), (A, Y)}

pendientes = []

F = {(A, Y)}

Extraemos (A, Y): [(A, Y)] → []. Con 0 llegamos a (A, Y) y con 1 a (B, Y). Ya están en R. No añadimos ni repetimos estados.

Recorrido del producto: comprobación final



R = {(A, X), (B, Y), (A, Y)}

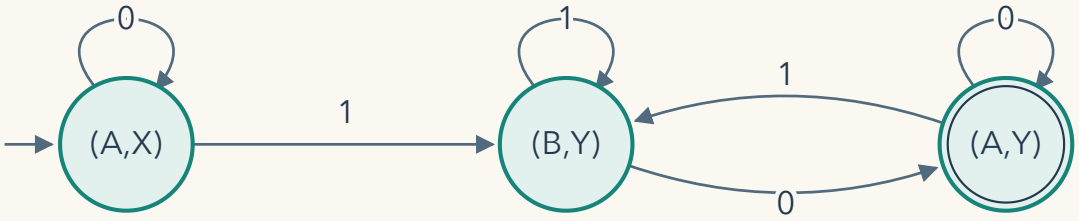
pendientes = []

F = {(A, Y)}

R ∩ F = {(A, Y)} ≠ ∅

pendientes = []: termina el bucle. Ahora comprobamos la intersección. Lenguajes distintos. La palabra 10 lleva a (A, Y).

La palabra 10 distingue los lenguajes

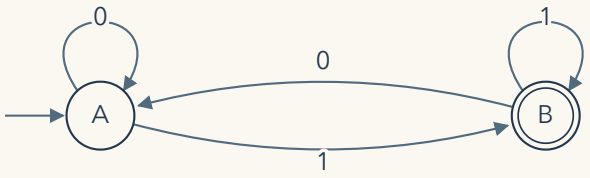


M_1 termina en A y **rechaza**. M_2 termina en Y y **acepta**.

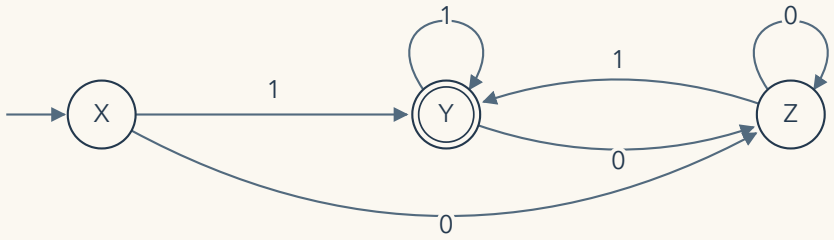
El producto acepta 10. Por tanto, $L(M_1) \neq L(M_2)$.

Mismo lenguaje, distinto número de estados

M1: dos estados

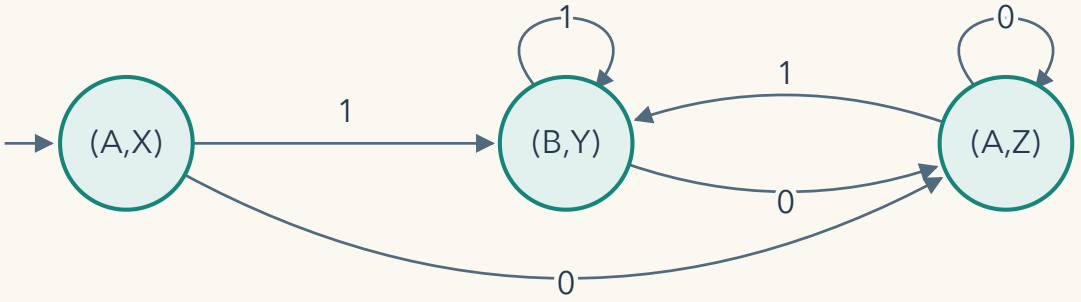


M2: tres estados



Los dos reconocen las palabras que terminan en 1. Podemos comprobarlo con el producto.

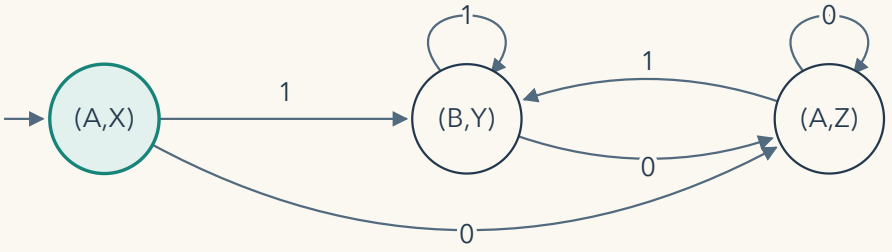
Un producto sin finales alcanzables



En (A, X) y (A, Z), ninguno es final. En (B, Y), ambos son finales.

Ninguna pareja es final del producto. F = ∅. Aplicamos ahora el procedimiento de vacío o no vacío al producto.

Recorrido del producto: inicialización



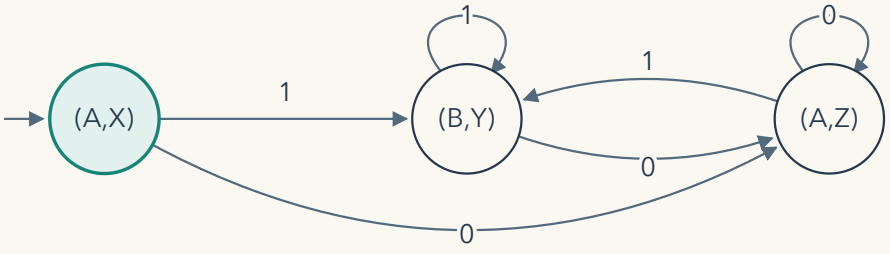
R = {(A, X)}

pendientes = [(A, X)]

F = ∅

Inicializamos R y pendientes con el estado inicial. La cola se atiende de izquierda a derecha.

Recorrido del producto: extraemos (A, X)



q = (A, X)

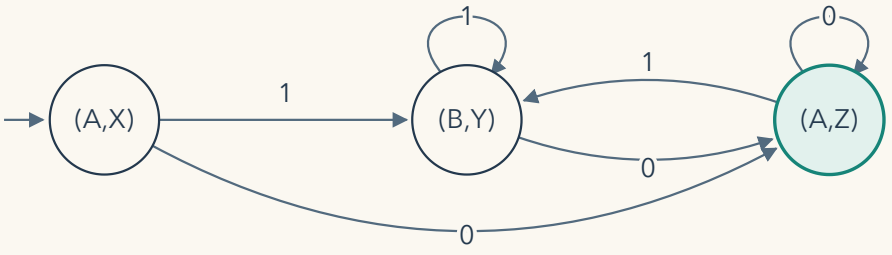
R = {(A, X), (A, Z), (B, Y)}

pendientes = [(A, Z), (B, Y)]

F = ∅

Extraemos (A, X): [(A, X)] → []. Con 0 llegamos a (A, Z) y con 1 a (B, Y). Añadimos los nuevos: {(A, Z), (B, Y)}.

Recorrido del producto: extraemos (A, Z)



q = (A, Z)

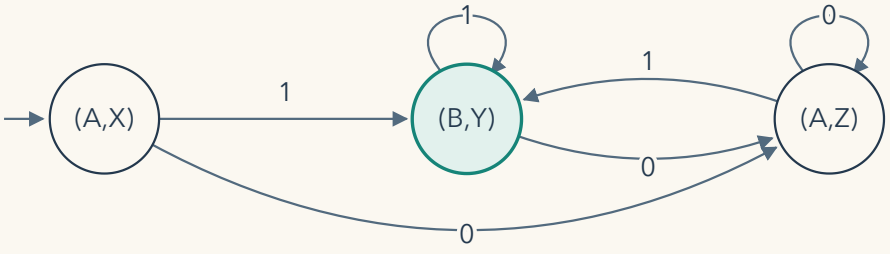
R = {(A, X), (A, Z), (B, Y)}

pendientes = [(B, Y)]

F = ∅

Extraemos (A, Z): [(A, Z), (B, Y)] → [(B, Y)]. Con 0 llegamos a (A, Z) y con 1 a (B, Y). Ya están en R. No añadimos ni repetimos estados.

Recorrido del producto: extraemos (B, Y)



q = (B, Y)

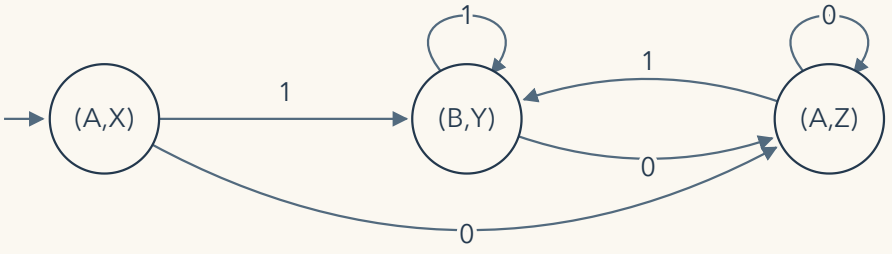
R = {(A, X), (A, Z), (B, Y)}

pendientes = []

F = ∅

Extraemos (B, Y): [(B, Y)] → []. Con 0 llegamos a (A, Z) y con 1 a (B, Y). Ya están en R. No añadimos ni repetimos estados.

Recorrido del producto: comprobación final



R = {(A, X), (A, Z), (B, Y)}

pendientes = []

F = ∅

R ∩ F = ∅

pendientes = []: termina el bucle. Ahora comprobamos la intersección. Lenguajes iguales.

Resumen de los procedimientos

01 **Vacío o no vacío.** Recorremos desde el inicial y comprobamos si algún final es alcanzable.

02 **Finito o infinito.** Conservamos los estados útiles y buscamos ciclos en el subgrafo resultante.

03 **Igualdad.** Construimos el producto con exactamente un componente final y comprobamos si su lenguaje es vacío.